



01

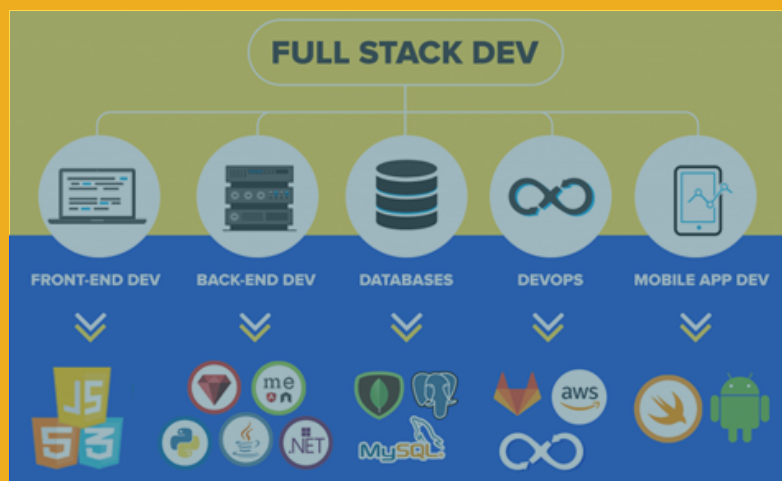
Introduction to ANVIL

Anvil

Anvil is a platform or framework for building full stack web applications with nothing but Python. Both server side and client side are in Python. Anvil is a web GUI tool where you can drag and drop elements to build your web application in a matter of minutes, plus you can deploy it immediately.

What is Full Stack?

Full stack web developers have the ability to design complete web applications and websites. They work on the frontend, backend, database and debugging of web applications or websites.



Front End (Client Side)

It is the visible part of website or web application which is responsible for user experience. The user directly interacts with the front-end portion of the web application or website. The front-end is built using a combination of technologies such as:

- **HTML:** Hyper Text Markup Language. It is used to design the front-end portion of web pages using markup language. HTML is the combination of Hypertext and Markup language. Hypertext defines the link between the web pages. The markup language is used to define the text documentation within tag which defines the structure of web pages.
- **CSS:** Cascading Style Sheets, fondly referred to as CSS, is a simply designed language intended to simplify the process of making web pages presentable. CSS allows you to apply styles to web pages. More importantly, CSS enables you to do this independent of the HTML that makes up each web page.
- **JavaScript:** JavaScript is a famous scripting language used to create the magic on the sites to make the site interactive for the user. It is used to enhancing the functionality of a website to running cool games and web-based software.

Back End (Server Side)

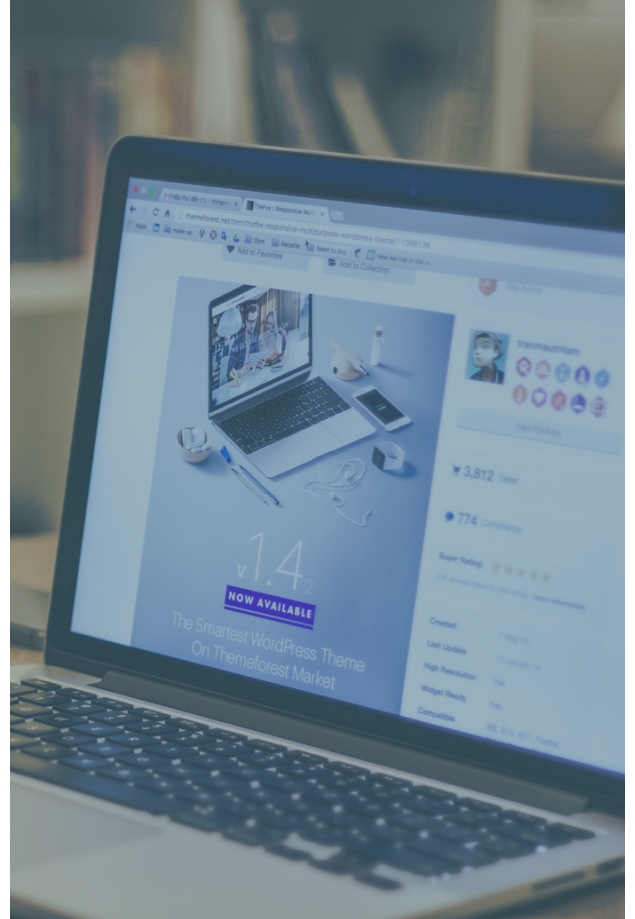
The back end, also called the server side, consists of the server which provides data on request, the application which channels it, and the database which organizes the information. The back-end portion is built by using some libraries, frameworks, and languages such as:

- JavaScript
- Ruby
- Python
- PHP
- C++
- Jawa

Database

Database is the collection of inter-related data which helps in efficient retrieval, insertion and deletion of data from database and organizes the data in the form of tables, views, schemas, reports etc.

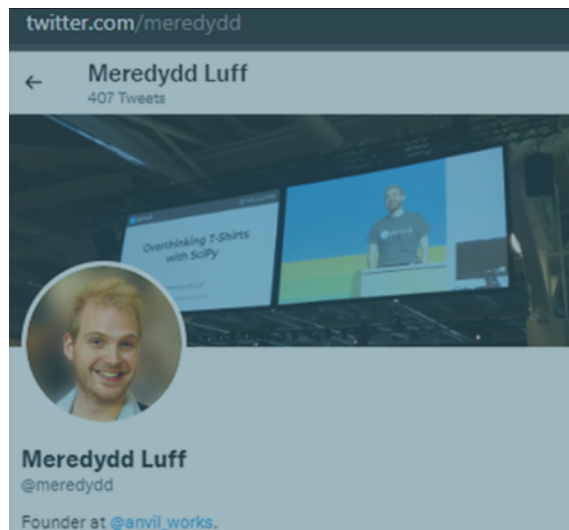
- Oracle: Oracle database is the collection of data which is treated as a unit. The purpose of this database is to store and retrieve information related to the query. It is a database server and used to manages information.
- MongoDB: MongoDB, the most popular NoSQL database, is an open-source document-oriented database. The term 'NoSQL' means 'non-relational'. It means that MongoDB isn't based on the table-like relational database structure but provides an altogether different mechanism for storage and retrieval of data.
- SQL: Structured Query Language is a standard Database language which is used to create, maintain and retrieve the relational database.



Why ANVIL?

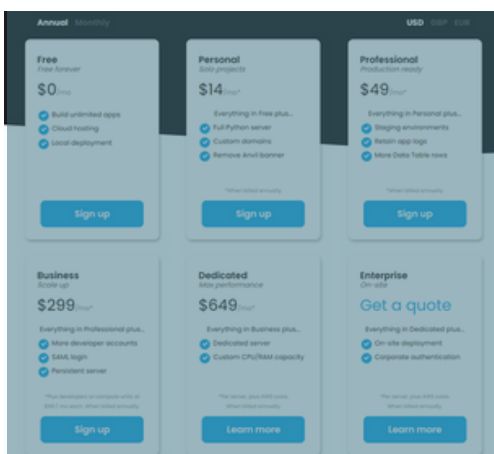
Anvil was founded by Meredydd who a software developer is living in Cambridge. The idea behind Anvil according to its founder, is that the way we create web applications is not efficient enough. For example suppose you have a MySQL database where you stored data in rows and columns, then you turn these data into objects on the server, they have properties and methods that can be accessed, and then you convert them from objects to JSON format so you can send them to Django rest framework or any other Rest framework using HTTP methods (GET, POST, PUT, DELETE).

On the other side of the HTTP connection there is JavaScript that is going to turn JSON in JavaScript object with other methods and properties, then we have to transform that into HTML DOM for the browser to render it and CSS to turn it into pixels. There are a lot of transformations here which are somehow repetitive and annoying, you have to learn a lot of programming languages and frameworks. Now with Anvil you can build a user interface with a simple drag and drop UI (or build it with code if you insist), plot with your favorite Python plotting library (Plotly, Matplotlib, etc.), and then deploy to the web in one click. No servers or containers to deal with.



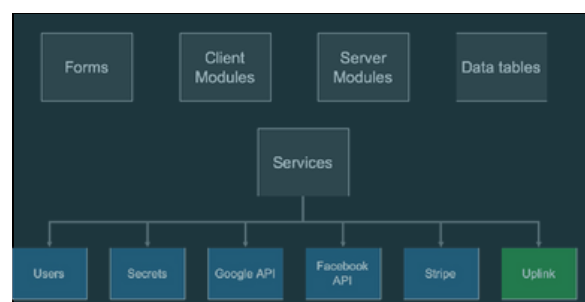
Create an Anvil Account

It is free to use Anvil, even for commercial purposes. However, certain features are only unlocked for users of one of paid plans. For now, all we need for our course is only the free plan. Go to <https://anvil.works/> to create an account.

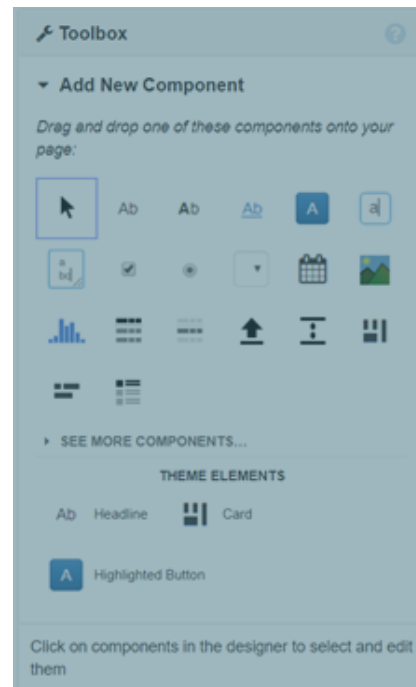


Anvil Building Blocks

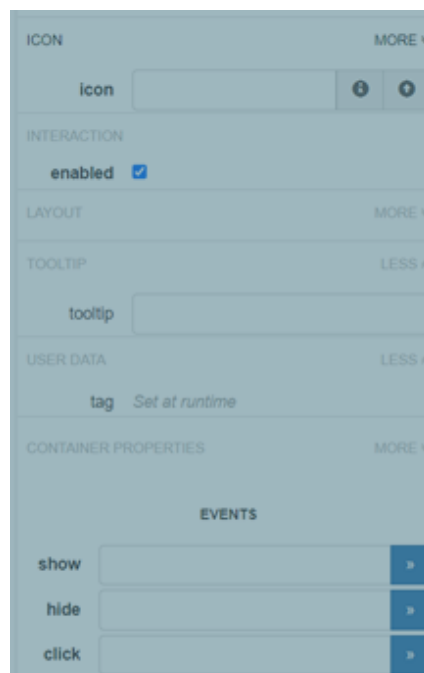
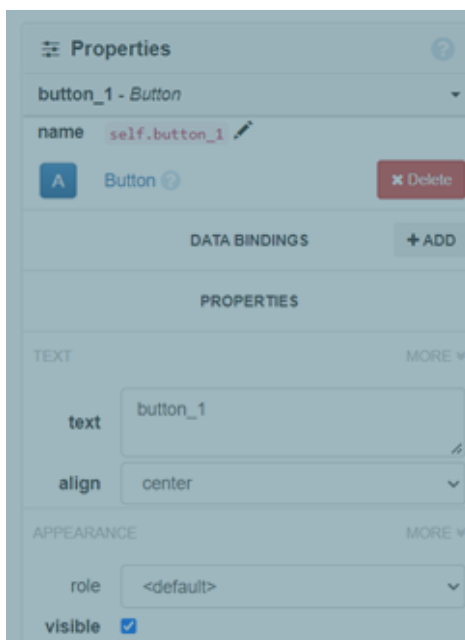
- **Forms.** These are basically the web pages which you design with a drag-and-drop designer.
- **Client Modules.** These modules are written and run in the web browser. We don't have to understand JavaScript, because Anvil will take care of it.
- **Server Modules.** Some codes need to run on a server and protected from the users messing with it, so we write it on the server side.
- **Data Tables.** Anvil has its own built-in databases called Data Tables. It's a system built on top of PostgreSQL.
- **Services.** There's a bunch of other services not explicitly stated for example sending and receiving email, etc. The uplink helps us connect code anywhere on the Internet to Anvil app using the Anvil Uplink.



Build User Interface (UI)



Adding components



Changing component properties

Client-side

The user interface of an Anvil app is programmed in Python. Anvil compiles that Python into Javascript, and when someone visits your app the client-side Python code runs in their web browser.

Client-side code interacts with your user interface by manipulating the Python objects that represent components. When components raise events, they call functions in your client-side code. (Try double click a button in your UI design to automatically create an event handler for button click).

```
def button_1_click(self, **event_args):  
    """This method is called when the button is clicked"""  
    pass
```

Everything is an object. Forms and components are represented in code as Python objects. We can instantiate Forms and components like any Python class. These Python objects have attributes and methods that we can use to control the UI.

```
label_instance = Label(text='Hello, World!')  
  
from .Form1 import Form1  
form1_instance = Form1(param='an_argument')  
  
# Change the text on the Label  
label_instance.text = "Hello again, World!"  
  
# Get a list of all the components in form1_instance  
components_on_form1 = [x for x in form1_instance.get_components()]
```

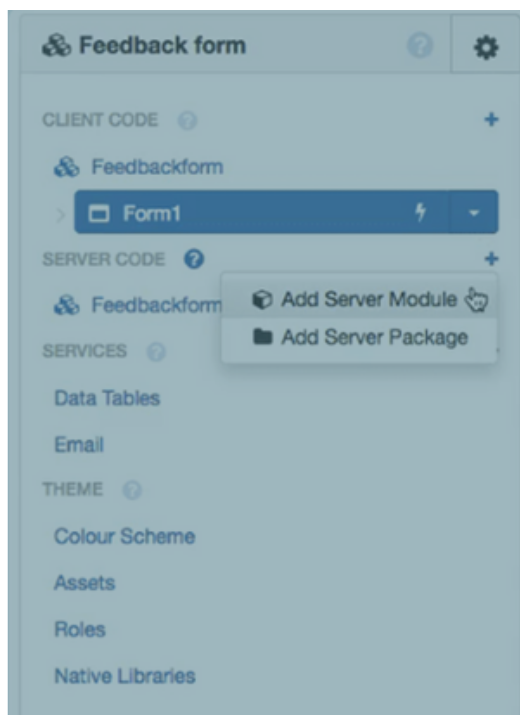


In the above picture Form1 (with lightning mark) will be open first at startup.

Server-side

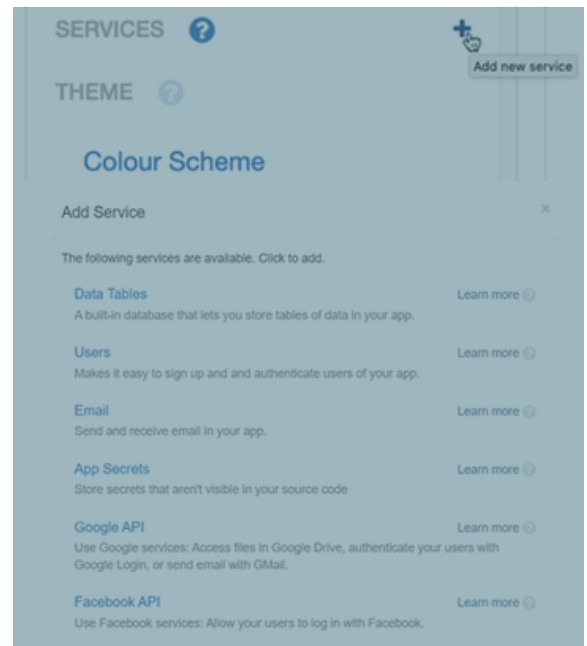
Anvil's Server Modules are a full server-side Python environment. Server Modules cannot be edited or seen by the user, so we can trust them to do what we tell them. This is why we use a Server Module to deal with database and media objects.

To make a server function available from your client code, decorate it as `@anvil.server.callable` then we can call it from client-side using `anvil.servel.call(<function_name>).`



Create Data Tables

Data Tables are an out-of-the-box option for data storage in Anvil, and they're backed by PostgreSQL. You can add Data Tables from Services.



Learning Anvil

Because we can't cover all the features in Anvil you can explore some more tutorials, documentation and examples of Anvil here <https://anvil.works/learn/>

Reference

- <https://towardsdatascience.com/an-introduction-to-anvil-full-stack-web-apps-with-nothing-by-python-cbab06392d13>
- <https://namelix.com/>
- <https://www.geeksforgeeks.org/what-is-full-stack-development/>
- <https://anvil.works/articles/full-stack-web-development>
- <https://training.talkpython.fm/player/course/anvil-web-apps-with-just-python/lecture/>
- <https://github.com/talkpython/anvil-course>

Picture Source

- <https://www.chitkara.edu.in/engineering/computer-science-engineering/specialisation-full-stack-development/>
- [pexels.com](https://www.pexels.com/)